Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

Jul 8, 2020 · 6 min read · ✨ · Listen

Save

SPACE SCIENCE WITH PYTHON

Space Science with Python — Density Estimators in the Sky

Part 18 of the tutorial series continues with the results from our last time. Since we know now how to determine the error-bars of an asteroid's position: How does it affect the sky coordinates?

4

The position determination of asteroids and stars is never perfect and contains e.g., measurement errors. Photo by Jeremy Perkins on Unsplash

Last time

Last week was Asteroid Day! A day that was introduced to remind us that cosmic threats are real … not only for the Dinosaurs, but also in recent decades like the Tunguska event in 1908 or the Meteor of Chelyabinsk in the year 2013.

Space Science with Python — Uncertain Movements of an Asteroid

Part 17 of the tutorial series. Today (30th June) is Asteroid Day and we will start to deep dive into the science of...

towardsdatascience.com

Observation surveys, follow up measurements, simulations, and so on are required to catalogue and understand our very cosmic vicinity. Currently, there are no larger objects on a direct collision course with our blue planet. However, observational errors propagate through to data and do not allow us to determine the position of an object with 100 % accuracy. The error-bars depend on the number of observations, observational conditions, total observation time, the distance to the object, its movement and brightness and other factors. It is a multi-dimensional error that can only be faced with *more and more and even more data*.

Luckily, 2020 JX1 is a *"good"* asteroid and the error-bars during the recent fly-by were quite small (considering cosmic scales).

But X, Y and Z coordinates do not help us at all … we need to set ecliptic, equatorial or azimuthal coordinates for our telescopes. Further, we have a solution space of Cartesian coordinates. *How can we translate this solution space to a proper ecliptic coordinate system function?* Let's find out!

Space Science with Python — Space maps

Part 5 of the tutorial series shows how to calculate and understand coordinate systems that are used for all upcoming...

towardsdatascience.com

Multidimensional Kernel Density Estimators

scikit-learn is a great resource for data science and machine learning algorithms. The library covers classifications, dimension reduction, as well as feature engineering and also clustering methods. The sophisticated documentation provides examples for miscellaneous use cases: One example covers the application of Kernel Density Estimators (KDEs) in spherical coordinates. Instead of the Euclidean metric, this example uses the so-called Haversine metric that is applied on the longitude and latitude values in radians:

Kernel Density Estimate of Species Distributions - scikit-learn 0.23.1 documentation

scikit-learn: machine learning in Python

scikit-learn.org

A KDE for ecliptic longitude and latitude coordinates appears suitable: Let's go.

For this tutorial, we use the already introduced libraries numpy, pandas, tqdm and maptlotlib. We then load the data that were created last time (the file is also part of the GitHub repository).

Part 1/8

Now, we convert the ecliptic longitude and latitude values to radians, using the numpy function *radians*.

Part 2/8

There are several ways to determine the optimal bandwidth for KDEs: cross validation, Grid Search or by using a simple rule-of-thumb like Scott's rule. We take a look at Scott's rule that scales with the number of samples, but also with the standard deviation. We compute the standard deviation of the longitude and latitude values:

The standard deviation of the longitude values is 50 % higher than for the latitude ones. Thus, the rule-of-thumb would result in 2 different bandwidths. Currently the scikit-learn KDE methods do not consider different bandwidths for different dimensions …

```
Standard deviation of the longitude in radians: 9.018716e-05
Standard deviation of the latitude in radians: 6.60280e-05
```

Are there other KDE implementations in Python? Luckily, yes! statsmodels provides several functions for statistics and statistical data exploration. The KDE section of statsmodels provides a multivariate version of a KDE:

statsmodels.nonparametric.kernel_density.KDEMultivariate - statsmodels

Edit description

www.statsmodels.org

The bandwidth computation can be done via several rules-of-thumb or cross validation. However, there is currently no Haversine metric support implemented.

Consequently both methods have their advantage and trade-offs:

- **scikit-learn's KDE**: Provides sophisticated input parameters with different metrics; but does not have a multi-dimensional bandwidth input
- **statsmodel's KDE**: Has a multivariate alternative; but does not contain other metrics.

The user needs to determine its requirements and goals. *Which trade-offs are acceptable? What does one need?* These are difficult questions to answer and as a (data) scientist one has to document every decision and consideration.

In our case, the spherical distortion should not affect our results severely. The data points are quite close to the ecliptic plane and the solution space is within a few arcminutes. We import the required statsmodels submodule in line 5, get the longitude and latitude values in line 8 and train the KDE model in line 11 to 13 using the the normal reference input.

Part 4/8

Let’s print the bandwidth results:

Part 5/8

```
Bandwidth longitude in radians (normal ref.): 3.02309e-05
Bandwidth latitude in radians (normal ref.): 2.21327e-05
```

Do we get different bandwidth results for different bandwidth determination methods?
Let's recompute the model with the cross validation (line 4) and print the results:

Part 6/8

Indeed, the resulting bandwidths for the longitude becomes smaller and the latitude one becomes larger! Both values are almost similar and one could consider

the Haversine method from scikit-learn. Anyway, we stay with the normal reference solution.

```
Bandwidth longitude in radians (cv_ml): 2.78430e-05
Bandwidth latitude in radians (cv_ml): 2.51614e-05
```

In a final step we would like to visualize the results in a plot. statsmodels provides a prediction function for our KDE model. But, we need a certain output format for the matplotlib function *imshow*. This function will allow us to visualise this 3 dimensional KDE (longitude, latitude plus probability density function (pdf)).

The computation needs to be done in "latitude slices". We iterate through the latitude solution space in a for-loop (line 8 following) and compute the pdf for the entire longitude solution space (line 15 to 19, computation in line 22). The final results are stored in a placeholder array (defined in line 4, stored in line 25). Finally, the array is converted to a numpy array (line 28). In line 31 we need to invert the resulting array, since we started with the minimum latitude value: however, matplotlib's *imshow* logic goes top-to-bottom.

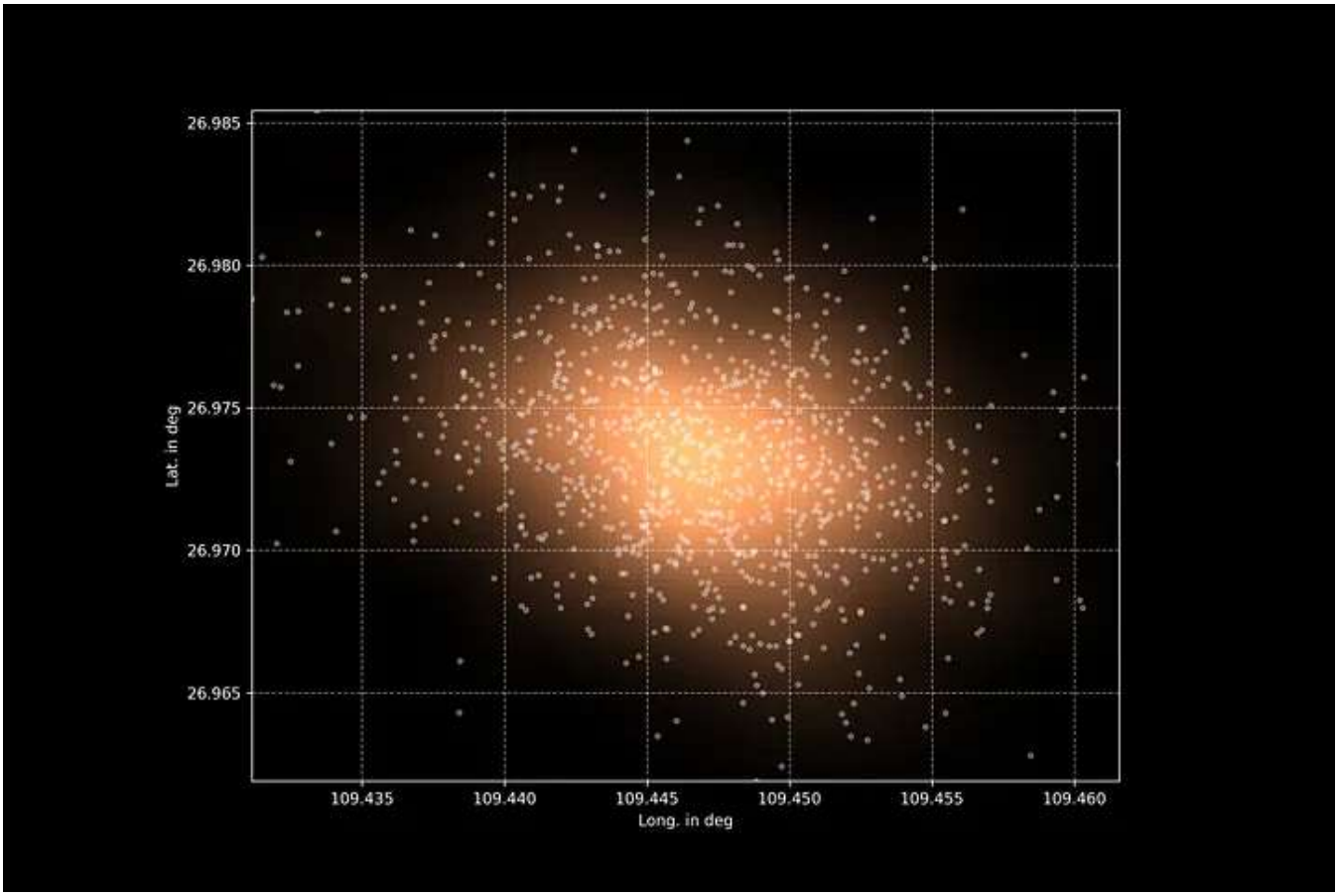
Part 7/8

Now we can plot the pdf and the original longitude/latitude data points as a scatter plot. A dark mode is set in line 2 and the figure is defined in line 5. The scatter plot of the data points is generated in line 8 to 10 and the *imshow* function in line 14 to 19 plots the KDE based pdf. Finally, we set a grid (line 22) and get rid off the scientific notation (line 25 and 26). Labels are set (line 29 and 30) and the resulting plot is stored in line 33. Let's take a look:

Part 8/8

The figure shows the ecliptic latitude vs. the ecliptic longitude in degrees. The white dots represent the original data: They represent possible sky coordinates of the

asteroid 2020 JX1 at midnight at the Asteroid Day 2020. The colour map in the background is the resulting KDE pdf. Brighter colours correspond to a higher appearance/detection probability than the darker areas. Thus, a follow-up telescope needs to cover this area of uncertainty at this particular time to detect the asteroid!



Ecliptic latitude vs. ecliptic longitude given in degrees. The white dots represent possible sky coordinates of the asteroid 2020 JX1 at midnight at the Asteroid Day 2020! The resulting area of uncertainty is based on a multivariate KDE and is colour coded in the background: a bright colour represents a higher probability.

Conclusion

Different KDE implementations have different advantages and disadvantages that were listed above. And although we did not consider the Haversine metric in our case, the resulting KDE pdf appears quite suitable. The discrete solution space scatter plot was transformed to an area of uncertainty. Within this area of uncertainty, a telescope operator needs to identify a moving object: our asteroid 2020 JX1! Considering the velocity vector of the asteroid, an apparent movement in the sky can be computed, too. But this task is left for you as a small programming assignment.

Please note that the area of uncertainty is quite small. There are far worse cases where telescope operators and astronomers have to look for a needle in a haystack. And we are not talking about a few, but hundreds of objects. Continuous operations and scientific work are required, now and in the future, to keep us and our home planet safe from unwanted stellar visitors.

Thomas

Python Space Science Data Science Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play